

C/C++ GoogleTest Generator VS Code Extension

This project is a **single VS Code plugin** that generates **GoogleTest** unit test files for C/C++ projects.

It supports two generation modes:

Mode	Description
internal	Uses your company internal AI REST API to generate better GoogleTest unit tests.
local	Uses no AI. Generates a basic GoogleTest skeleton locally.

This plugin does **not** require OpenAI and does not include OpenAI code.

What This Plugin Does

The plugin reads the selected C/C++ code or the full current file, then generates a GoogleTest file under the configured test folder.

Example output path:

`tests/calculator_test.cpp`

Supported source files:

`.c`
`.h`
`.cpp`
`.cc`
`.cxx`
`.hpp`
`.hxx`

Final Architecture

VS Code Plugin

```
|  
+-- Command: C/C++ Unit Test: Generate Tests  
|  
+-- Mode: internal
```

```

|      +-- Send C/C++ code to company internal AI API
|      +-- Receive generated GoogleTest code
|      +-- Write tests/<source>_test.cpp
|
+-- Mode: local
    +-- Parse functions locally
    +-- Generate basic GoogleTest skeleton
    +-- Write tests/<source>_test.cpp

```

This is **one plugin** with two modes, not multiple plugins.

Project Structure

```

vs_plugin/
|-- package.json
|-- tsconfig.json
|-- README.md
|-- src/
|   |-- extension.ts
|   |-- internalAiClient.ts
|   |-- localSkeletonGenerator.ts
|   |-- promptBuilder.ts
|   |-- testWriter.ts
|   `-- fileUtils.ts
`-- .vscode/
    |-- launch.json
    `-- tasks.json

```

Prerequisites

Install Node.js and npm, then install the VS Code extension tooling:

```
npm install -g yo generator-code vsce
```

Inside the project folder:

```
npm install
```

Build the Plugin

```
npm run compile
```

Run Locally in VS Code

1. Open the project folder in VS Code.
2. Press F5.
3. A new **Extension Development Host** window opens.
4. Open a `.c`, `.h`, `.cpp`, `.cc`, `.cxx`, `.hpp`, or `.hxx` file.
5. Open Command Palette with `Ctrl + Shift + P`.
6. Run one of these commands:

```
C/C++ Unit Test: Generate Tests
C/C++ Unit Test: Generate with Internal AI
C/C++ Unit Test: Generate Local Skeleton
C/C++ Unit Test: Set Internal AI API Key
C/C++ Unit Test: Clear Internal AI API Key
```

Configuration

Open VS Code `settings.json`.

Local Mode - No AI

Use this when you do not want the plugin to call any AI service.

```
{
  "cppUnitTest.generationMode": "local",
  "cppUnitTest.testFolder": "tests",
  "cppUnitTest.overwriteExistingFile": false
}
```

Internal Company AI Mode

Use this when you want the plugin to call your company internal AI API.

```
{
  "cppUnitTest.generationMode": "internal",
  "cppUnitTest.internalApiBaseUrl": "https://your-company-ai-server.example.com",
  "cppUnitTest.internalModel": "company-coder-model",
  "cppUnitTest.testFolder": "tests",
  "cppUnitTest.overwriteExistingFile": false
}
```

Internal AI API Contract

When `generationMode` is set to `internal`, the plugin sends a POST request to:

```
<internalApiBaseUrl>/generate-unit-tests
```

Example request body:

```
{
  "model": "company-coder-model",
  "framework": "gtest",
  "fileName": "calculator.cpp",
  "language": "C++",
  "sourceCode": "int add(int a, int b) { return a + b; }",
  "prompt": "Generate GoogleTest unit tests..."
}
```

The internal API should return one of these fields:

```
{
  "code": "#include <gtest/gtest.h>\n..."
}
```

or:

```
{
  "text": "#include <gtest/gtest.h>\n..."
}
```

or:

```
{
  "result": "#include <gtest/gtest.h>\n..."
}
```

Internal API Key

If your internal company AI API requires a bearer token, run this command from the Command Palette:

C/C++ Unit Test: Set Internal AI API Key

The key is stored with VS Code SecretStorage.

If your internal API does not require a token, skip this step.

GoogleTest for C++ Example

Input file:

```
#include "calculator.h"

int add(int a, int b) {
```

```
    return a + b;
}
```

Generated local GoogleTest skeleton:

```
#include <gtest/gtest.h>
#include "calculator.h"

TEST(CalculatorTest, AddShouldWork) {
    // TODO: arrange

    auto result = add(0, 0);

    // TODO: assert
    // EXPECT_EQ(expected, result);
}
```

GoogleTest for C Example

Input file:

```
#include "math_utils.h"

int add(int a, int b) {
    return a + b;
}
```

Generated local GoogleTest skeleton:

```
#include <gtest/gtest.h>

extern "C" {
#include "math_utils.h"
}

TEST(MathUtilsTest, AddShouldWork) {
    // TODO: arrange

    auto result = add(0, 0);

    // TODO: assert
    // EXPECT_EQ(expected, result);
}
```

For C source code, the generated GoogleTest file is still `.cpp`, but the C header is wrapped with `extern "C"`.

Package as a VSIX

`npm run package`

Install locally:

`code --install-extension cpp-unit-test-generator-0.0.1.vsix`

Push to GitHub

```
git clone https://github.com/Qijiang60/vs_plugin.git
cd vs_plugin
```

Copy all project files into this repo folder.

```
npm install
npm run compile
```

```
git add .
git commit -m "Add C/C++ GoogleTest generator VS Code extension"
git push origin main
```

If your repo uses master instead of main:

```
git push origin master
```

Recommended First Test

Create a file named `calculator.cpp`:

```
#include "calculator.h"

int add(int a, int b) {
    return a + b;
}

bool isPositive(int value) {
    return value > 0;
}
```

Run:

C/C++ Unit Test: Generate Local Skeleton

Expected generated file:

`tests/calculator_test.cpp`

Notes

- Use `local` mode when you do not want any code sent outside VS Code.
- Use `internal` mode only with a company-approved AI endpoint.
- The plugin is GoogleTest-only by design.
- You can add Unity or CppUTest later, but this MVP keeps the scope simple.